

Computer Networks Assignment

Student Name: Shagun Goyal

UID:24BDA70214

Branch: AIT-CSE

Section/Group: 24AIT KRG-2

Subject Name: Computer Networks

Subject Code: 24CSH-337

Question 1: IP Addressing, Classes, Subnetting & Supernetting

What is an IP Address?

An IP (Internet Protocol) address is a unique 32-bit numerical address assigned to a device connected to a network using the TCP/IP protocol. It works at **Layer 3 (Network Layer)** of the OSI model.

An IP address mainly performs two functions:

1. **Host identification:** It identifies a particular device on a network.
2. **Logical addressing and routing:** It helps routers determine where data should be sent so that it reaches the correct destination network and device.

Why Do We Need Classes in IPv4?

In the original **classful IPv4 addressing system**, IP addresses were divided into five classes: **A, B, C, D, and E**.

The purpose of these classes was to allocate IP addresses according to the size and requirements of an organization:

- **Class A** was designed for very large networks requiring millions of host addresses.
- **Class B** was intended for medium to large organizations, such as universities.
- **Class C** was suitable for smaller organizations and LANs requiring fewer host addresses.
- **Class D** was reserved for multicast communication.
- **Class E** was reserved for experimental and research purposes.

Each class had a predefined boundary between the **Network ID** and **Host ID**. This allowed routers to determine the destination network using the default class-based addressing scheme.

Explanation of IPv4 Classes

An IPv4 address contains **32 bits**, divided into four octets of 8 bits each. The octets are separated by dots, for example, **192.168.1.1**.

Class	First Octet Bits (Leading)	First Octet Range	Default Subnet Mask	Network ID Bits	Host ID Bits	Total Networks	Usable Hosts per Network	Primary Purpose
A	0	1–126	255.0.0.0	8	24	126	16,777,214	Extremely large networks
B	10	128–191	255.255.0.0	16	16	16,384	65,534	Medium-to-large networks
C	110	192–223	255.255.255.0	24	8	2,097,152	254	Small Local Area Networks (LANs)
D	1110	224–239	N/A	N/A	N/A	N/A	N/A	Multicasting
E	1111	240–255	N/A	N/A	N/A	N/A	N/A	Reserved for research and future use

Important Notes on Host Calculations

In a normal IPv4 network, two addresses are reserved:

1. **Network Address:** The first address in the network, where all host bits are 0. It identifies the network itself.
2. **Broadcast Address:** The last address in the network, where all host bits are 1. It is used to send data to all hosts on that network.

Therefore, the number of usable hosts is generally:

$$\text{Usable Hosts} = 2^h - 2$$

where **h** is the number of host bits.

In Class A addressing, **127.0.0.0/8** is reserved for loopback testing. Therefore, only **126 Class A network numbers** are available for normal addressing.

Subnetting and Supernetting

1. Subnetting

Subnetting is the process of dividing one large network into several smaller logical networks called **subnets**. It is achieved by borrowing bits from the host portion and using them as additional network/subnet bits.

Purpose of Subnetting

- Reduces network congestion by creating smaller broadcast domains.
- Improves network security and organization.
- Makes IP address management more efficient.

- Allows a large network to be divided according to departmental or geographical requirements.

Example

Consider the Class C network:

192.168.1.0/24

The default subnet mask is:

255.255.255.0

There are **8 host bits**.

Suppose we borrow **2 bits** from the host portion.

- New subnet mask = **/26**
- New subnet mask in decimal = **255.255.255.192**
- Number of subnets = $2^2 = 4$
- Remaining host bits = **6**
- Usable hosts per subnet = $2^6 - 2 = 62$

Resulting Subnets

1. **Subnet 1:** 192.168.1.0/26
Usable IP range: **192.168.1.1 – 192.168.1.62**
2. **Subnet 2:** 192.168.1.64/26
Usable IP range: **192.168.1.65 – 192.168.1.126**
3. **Subnet 3:** 192.168.1.128/26
Usable IP range: **192.168.1.129 – 192.168.1.190**
4. **Subnet 4:** 192.168.1.192/26
Usable IP range: **192.168.1.193 – 192.168.1.254**

Thus, one **/24 network** is divided into **four /26 subnets**, with **62 usable hosts in each subnet**.

2. Supernetting

Supernetting, also known as **route aggregation** or **CIDR aggregation**, is the opposite of subnetting. It combines multiple smaller, contiguous networks into one larger network.

Purpose of Supernetting

The main purpose is to **reduce the size of routing tables**. Instead of maintaining separate routing entries for several networks, a router can use a single summarized route.

Example

Suppose an organization owns these four contiguous Class C networks:

1. 192.168.0.0/24
2. 192.168.1.0/24
3. 192.168.2.0/24
4. 192.168.3.0/24

The third octets in binary are:

- 0 = **00000000**
- 1 = **00000001**

- 2 = 00000010
- 3 = 00000011

The first **22 bits** are common to all four networks.

Therefore, the four /24 networks can be summarized as:

Supernet Address = 192.168.0.0/22

Supernet Mask = 255.255.252.0

Thus, four separate networks can be represented by a **single routing entry**.

Question 2: Link State vs. Distance Vector Routing

Comparison Table

Parameter	Distance Vector Routing	Link State Routing
Algorithm Used	Bellman-Ford Algorithm	Dijkstra's Shortest Path First (SPF) Algorithm
Topology Knowledge	Knows information about its neighboring routers and their reported distances	Maintains a complete view of the network topology
Update Mechanism	Periodically sends routing information to neighboring routers	Floods Link State Advertisements (LSAs) when topology changes
Convergence Speed	Generally slower and can suffer from routing loops and count-to-infinity problems	Generally faster and responds quickly to topology changes
Resource Overhead	Lower CPU and memory requirements, but periodic updates consume bandwidth	Higher CPU and memory requirements for maintaining the topology database and running SPF
Example Protocols	RIP and EIGRP	OSPF and IS-IS

How Algorithms Identify Least Cost Path (With Examples)

1. Distance Vector Routing (Bellman-Ford Algorithm)

In Distance Vector routing, each router maintains a table containing information such as:

Destination, Cost, and Next Hop

Routers exchange their distance-vector information with directly connected neighbors. Each router then calculates the least-cost path using the **Bellman-Ford equation**:

$$D_x(y) = \min_v \{ c(x,v) + D_v(y) \}$$

where:

- $D_x(y)$ = estimated cost from router x to destination y.
- $c(x,v)$ = cost of the link from router x to neighboring router v.
- $D_v(y)$ = cost from neighbor v to destination y.

Step-by-Step Example

Consider three connected routers **A, B, and C**.

Suppose the direct link costs are:

- $A \rightarrow B = 1$
- $B \rightarrow C = 2$
- $A \rightarrow C = 5$

Initially, router A knows:

- Distance to B = **1**
- Distance to C = **5**

Router B advertises its routing information to A. B reports that its distance to C is **2**.

Router A can therefore calculate an alternative route to C:

$$A \rightarrow B \rightarrow C = 1 + 2 = 3$$

Since **3 < 5**, router A selects the route through B.

Therefore:

- Shortest path from A to B = $A \rightarrow B$, cost = **1**
- Shortest path from A to C = $A \rightarrow B \rightarrow C$, cost = **3**

The routing table at A is updated accordingly.

2. Link State Routing (Dijkstra's Algorithm)

In Link State routing, every router collects Link State Advertisements (LSAs) from other routers. These advertisements allow each router to build a **complete representation of the network as a graph**.

Each router then uses **Dijkstra's Shortest Path First (SPF) algorithm** to calculate the shortest paths from itself to all other destinations.

Dijkstra's Algorithm Notation

- **D(v)**: Current cost of the least-cost path from the source to destination v.
- **p(v)**: Predecessor node on the current shortest path to v.
- **N'**: Set of nodes whose least-cost paths have been permanently determined.

Step-by-Step Example

Consider a network containing routers **A, B, C, and D** with link costs:

- $A \rightarrow B = 1$
- $A \rightarrow C = 4$
- $B \rightarrow C = 2$
- $B \rightarrow D = 5$
- $C \rightarrow D = 1$

Assume **A** is the source router.

1. Initialization

Set:

- $D(A) = 0$
- $D(B) = 1$

- $D(C) = 4$
- $D(D) = \infty$

Initially, only A is in the set of permanently known nodes.

2. Iteration 1

Select **B**, because it has the smallest tentative cost of **1**.

Add B to the set.

Update the costs through B:

- Cost to C = $\min(4, 1 + 2) = 3$
- Cost to D = $\min(\infty, 1 + 5) = 6$

3. Iteration 2

Select **C**, because its current cost is **3**.

Add C to the set.

Update D through C:

- Cost to D = $\min(6, 3 + 1) = 4$

4. Iteration 3

Select **D**, whose cost is **4**.

All nodes have now been processed, so the algorithm terminates.

Outcome

The shortest paths from A are:

- To B: $A \rightarrow B$, Cost = 1
- To C: $A \rightarrow B \rightarrow C$, Cost = 3
- To D: $A \rightarrow B \rightarrow C \rightarrow D$, Cost = 4

Question 3: Congestion Control, Traffic Shaping & Fragmentation

What is Network Congestion & Why Control It?

Network congestion occurs when the amount of data entering a network is greater than the network's ability to process or transmit it. When routers and links become overloaded, their buffers may fill up, resulting in packet loss and increased delays.

Reasons We Need Congestion Control

1. Prevent Packet Loss and Buffer Overflow:

When router queues become full, packets may be discarded. This can cause transport protocols such as TCP to retransmit the lost packets.

2. Avoid Congestion Collapse:

Excessive retransmissions can consume the remaining network capacity. As congestion increases, useful throughput can fall significantly.

3. Reduce Latency and Jitter:

Heavy congestion creates long queues, which increase delay and variation in delay. This can negatively affect real-time applications such as VoIP and video conferencing.

Traffic Shaping: Leaky Bucket vs. Token Bucket

Both **Leaky Bucket** and **Token Bucket** algorithms are used to control network traffic. However, they handle bursts in different ways.

Feature	Leaky Bucket Algorithm	Token Bucket Algorithm
Primary Goal	Produces a smooth and nearly constant output rate	Controls the average rate while allowing controlled bursts
Burst Handling	Bursty traffic is smoothed; traffic above the allowed rate may be queued or dropped	Tokens can accumulate during idle periods, allowing later bursts
Output Rate	Generally fixed or constant	Variable; can temporarily increase during bursts
Overflow Action	Excess packets may be discarded when the bucket is full	Excess tokens are discarded when the bucket is full; packets can still be transmitted when sufficient tokens are available

Numerical Examples

Leaky Bucket Example

Suppose:

- Bucket Capacity = **10 KB**
- Fixed Output (Leak) Rate = **2 KB/s**
- A burst of **10 KB** arrives at time **t = 0**
 1. The **10 KB** burst fits exactly into the bucket.
 2. The bucket releases data at a constant rate of **2 KB/s**.
 3. The complete burst takes:

Time = $10 \text{ KB} \div 2 \text{ KB/s} = 5 \text{ seconds}$

Therefore, the burst is transmitted smoothly over **5 seconds**.

Token Bucket Example

Suppose:

- Bucket Capacity = **5 tokens**
- Token Generation Rate = **1 token/s**
- Each token allows the transmission of **1 packet**
 1. During an idle period, the bucket fills to its maximum capacity of **5 tokens**.
 2. At a later time, a sudden burst of **5 packets** arrives.
 3. Since 5 tokens are available, all **5 packets can be transmitted immediately**.
 4. The five tokens are consumed, leaving the bucket empty.

This demonstrates how the Token Bucket algorithm allows controlled bursts while maintaining an average transmission rate.

Why Do We Need Fragmentation?

Fragmentation is a network-layer process in which a large IP datagram is divided into smaller fragments so that it can travel across a network link whose **Maximum Transmission Unit (MTU)** is smaller than the original packet.

Reasons for Fragmentation

1. **Different Link Technologies:**

Different network technologies may support different MTU sizes. For example, standard Ethernet commonly uses an MTU of **1500 bytes**, while other links may support smaller or different MTU values.

2. **Interoperability:**

Packets may travel through multiple networks with different MTU limits. Fragmentation allows a large packet to be divided into smaller pieces when necessary so that it can cross a link with a smaller MTU.

IP Header Fields Supporting Fragmentation

The IPv4 header contains three important fields related to fragmentation:

1. **Identification**

The **Identification field** is 16 bits long. The sender assigns an identification value to an IP datagram. All fragments belonging to the same original datagram carry the same identification value, allowing the destination to reassemble them.

2. **Flags**

The **Flags field** contains three bits, of which two are important for fragmentation:

- **DF (Don't Fragment):** When set to 1, routers are not allowed to fragment the packet. If the packet is too large for the outgoing link, it is discarded and an ICMP error can be generated.
- **MF (More Fragments):** This bit is set to 1 for all fragments except the final fragment. The last fragment has **MF = 0**.

3. **Fragment Offset**

The **Fragment Offset** field is 13 bits long. It specifies the position of a fragment's data relative to the beginning of the original IP datagram.

The offset is measured in units of **8-byte blocks**. This information enables the destination device to correctly reassemble all fragments into the original datagram.